Trabalho Prático de Redes de Computadores II - CI1061 2026/2

Prof. Giovanni Venâncio

1 Sistema de Balanceamento de Cargas

1.1 Visão Geral

Sua tarefa é desenvolver um sistema de balanceamento de cargas, também conhecido como *Load Balancer* (LB). Um LB é uma entidade intermediária entre os clientes e um conjunto de servidores. O objetivo é distribuir conexões de forma a evitar a sobrecarga de um único servidor. Um LB possui outras vantagens como, por exemplo, evitar pontos únicos de falha.

Para distribuir as conexões entre os servidores, um LB pode implementar diversas políticas. Uma delas é a *Round-Robin*, onde as conexões são distribuídas de forma sequencial e circular entre os servidores disponíveis, garantindo uma quantidade de conexões equivalente entre os servidores. Outra possível política é a *Least Connections*, onde uma nova conexão é direcionada para o servidor com a menor quantidade de conexões abertas no momento. A Figura 1 ilustra o sistema descrito.

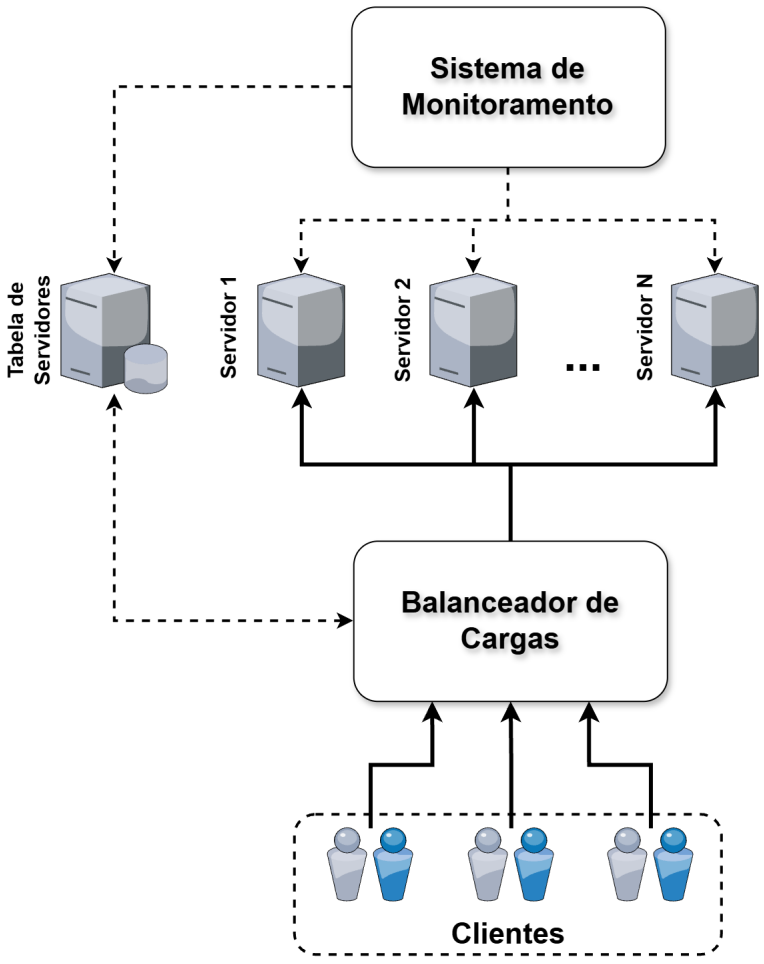


Figura 1: Sistema de balanceamento de cargas. As linhas contínuas representam o fluxo de dados das conexões (dados efetivos transmitidos). As linhas tracejadas representam mensagens de controle e monitoramento.

O sistema deve ser composto pelos seguintes componentes:

1. **Clientes.** Múltiplos clientes que desejam se comunicar com um determinado serviço (executado por algum dos servidores). Clientes conhecem somente o endereço IP e porta do LB. Portanto, conectam-se **somente** com o LB. Um cliente permanece conectado e enviando dados durante um intervalo específico de tempo.

2. ***Load Balancer.*** Entidade responsável por intermediar as conexões dos clientes com os servidores. A cada nova conexão recebida, escolhe um dos N servidores para atender a conexão de acordo com uma política específica.

3. **Servidores.** Servidores destino que recebem o tráfego dos clientes (através do LB). Os servidores não precisam implementar nenhuma funcionalidade e podem simplesmente retornar qualquer mensagem de confirmação. **Atenção:** servidores estão sujeitos a falhas!

4. **Sistema de Monitoramento.** Sistema que verifica de maneira periódica quais servidores estão funcionando corretamente e quais falharam.

5. **Tabela de servidores.** Tabela que mantém informações sobre os servidores. O sistema de monitoramento atualiza essa tabela para manter a informação de quais servidores estão operantes, enquanto que o balanceador de cargas precisa consultar essa tabela no momento em que irá executar a política de escolha de servidor.

1.2 Funcionalidades

O sistema descrito na Seção 1.1 deverá implementar as funcionalidades descritas abaixo.

1. **Envio de dados do cliente.** Implementar um *script* que simule um conjunto de M clientes, onde cada cliente executa durante K segundos. O *script* deve, obrigatoriamente, gerar carga o suficiente para que o balanceador de cargas distribua as conexões de acordo com a política de escolha.

2. ***Load Balancer (LB).*** Ao receber uma nova conexão de um cliente, seleciona um servidor para atender essa conexão. Deve implementar duas políticas de escolha: *(i) Round-Robin*; *(ii) Least Connections*. Apenas uma política de escolha deve permanecer ativa por vez, definindo durante a inicialização do LB.

   A implementação do LB deve ser concorrente, permitindo que múltiplas requisições possam ser atendidas de maneira não bloqueante. A cada nova conexão recebida, o LB deve selecionar o servidor correspondente e estabelecer uma conexão com este servidor. O cliente então irá enviar dados ao LB que irá repassar para o servidor.

3. **Servidores.** Recebe solicitações de conexões do LB e, posteriormente, o fluxo de dados de um determinado cliente. A implementação do servidor também deve ser concorrente, permitindo que o servidor atenda múltiplos clientes de maneira não bloqueante. O servidor não precisa implementar nenhuma lógica – pode simplesmente enviar um `OK` como resposta. A resposta é enviada ao LB que, por sua vez, encaminha para o respectivo cliente.

4. **Sistema de Monitoramento.** Deve monitorar os N servidores, verificando se ainda estão ativos. O monitoramento deve atualizar a tabela de servidores de forma que o LB saiba quais (e quantos) servidores estão ativos no momento. Caso o sistema de monitoramento detecte que um servidor falhou, deve atualizar a tabela de servidores.

O trabalho também deve implementar o tratamento de falhas, recuperando servidores que em algum momento falharam. A estratégia e forma de implementar a recuperação fica a critério da equipe. A recuperação de um servidor também deve atualizar a tabela de servidores.

5. **Tabela de Servidores.** Mantém informações relevantes sobre o conjunto de servidores. As seguintes informações devem estar presentes:

   - Número total de conexões (mantidas pelo LB);
   - Quantidade de conexões de cada servidor;
   - Endereço IP e Porta de cada servidor;
   - Status de cada servidor (UP/DOWN);

   Tanto o sistema de monitoramento quanto o LB interagem com essa tabela para obter e/ou atualizar valores.

1.3 Detalhes Técnicos

- Deve ser possível especificar a quantidade N de servidores.
- Deve ser possível especificar a política do LB no momento de sua execução.
- O cliente deve saber somente o endereço IP e porta do LB. É estritamente proibido o cliente conectar diretamente com os servidores.
- A forma de implementar a tabela de servidores é livre.
- Os trabalhos devem ser capazes de serem executados nos servidores do DInf.
- A implementação deve ser feita utilizando *sockets* TCP. Não será permitida a utilização de bibliotecas de rede adicionais além das que vimos em aula. Os arquivos exemplo de cliente e servidor TCP (em C e em Python) estão disponíveis no Moodle.
- A implementação poderá ser feita em qualquer linguagem de programação (*e.g.,* C, Python, etc.).

1.4 Dicas

- A política *Least Connections* pode ser testada em situações onde a duração das conexões do cliente varia;
- Para a implementação concorrente (no LB e nos servidores), pode-se utilizar múltiplas *threads* **ou** múltiplos processos independentes;
- O *script* dos clientes pode implementar várias técnicas para simular uma carga real de dados. Exemplos: inicializar múltiplos clientes em paralelo, definir um tempo arbitrário em que cada cliente permanece conectado enviando dados, inserir tempo de espera entre execuções de clientes (*i.e., sleep()*).
- Crie um *log* que armazene detalhes da execução do sistema, permitindo a depuração e análise durante a defesa do trabalho.

2 Regras Gerais de Entrega e Avaliação

Além da implementação do sistema será necessário desenvolver um relatório. O relatório deve conter uma descrição completa do sistema, detalhando a forma como foi desenvolvido, além das decisões de projeto tomadas. O relatório deve ser desenvolvido em LaTeX e deve conter no máximo 4 páginas.

O trabalho deverá ser feito em equipes de **três** integrantes. A entrega deverá ser feita através do Moodle da disciplina. Outras formas de envio não serão consideradas. Somente um integrante da equipe deve fazer o envio. O envio de trabalhos fora do prazo não serão considerados.

A entrega deve consistir dos seguintes itens:

- Arquivos fonte do trabalho (compressão em .tgz);
- Relatório do trabalho em PDF.

O não envio de algum desses itens levará nota zero. A cópia do trabalho (plágio) não será tolerada e acarretará em nota igual a **zero** para todos os envolvidos. Trabalhos desenvolvidos com o auxílio de Inteligência Artificial (IA) também receberão nota **zero**.

O trabalho deverá ser defendido por todos os integrantes da equipe. O cronograma de defesa será divulgado via Moodle. A nota da defesa é **individual** para cada integrante da equipe. O não comparecimento na defesa do trabalho resultará em nota **zero**. Além disso, a nota de defesa é multiplicativa com a nota da implementação e relatório, *i.e.,* caso o trabalho esteja funcionando, mas a nota da defesa for zero, a nota final será zero.

Durante a defesa, é esperado que a equipe tenha domínio sobre o trabalho desenvolvido, e que esteja preparada para demonstrar o funcionamento do trabalho. **IMPORTANTE:** prepare arquivos de testes e/ou cenários onde seja possível avaliar todos os aspectos do trabalho.